



U.S. General Services
Administration



Agentic AI Book Club - Session 2

Reviewing Chapter 1.2 & 1.3

US General Service Administration
AI Community of Practice - March 2026

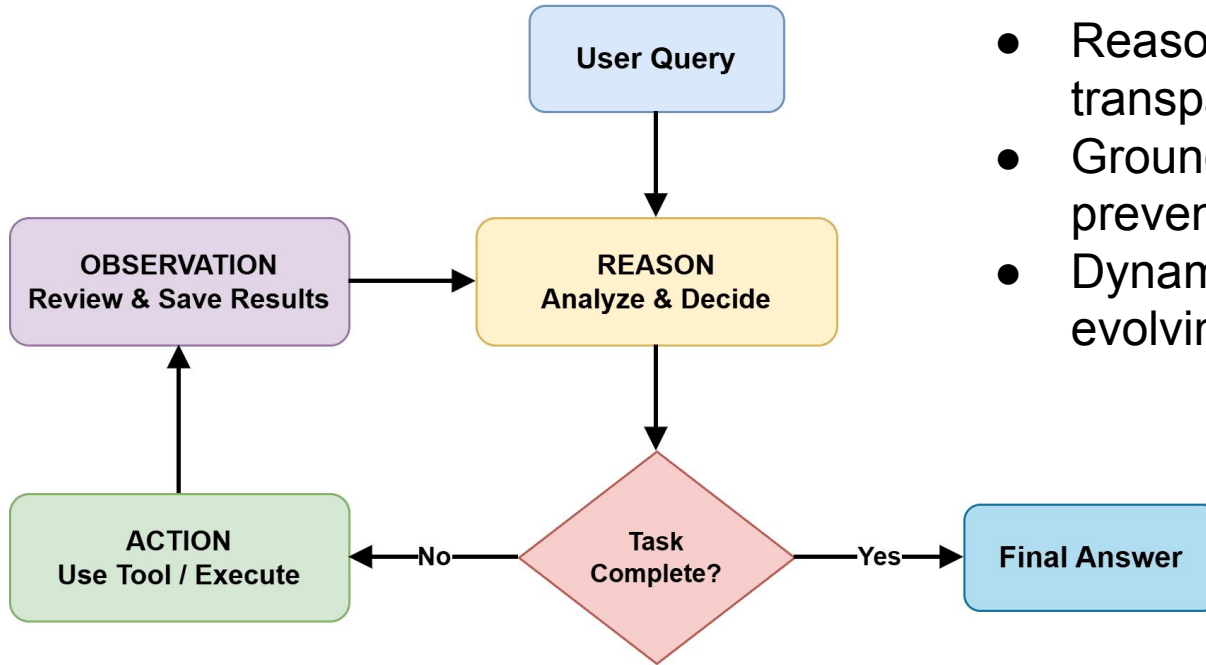
Agenda

- Housekeeping
 - Chapter and quiz review
 - **Sharing your successes and challenges so far**
 - Critical recommendation*
 - Intro to next week's chapters
- 

Housekeeping

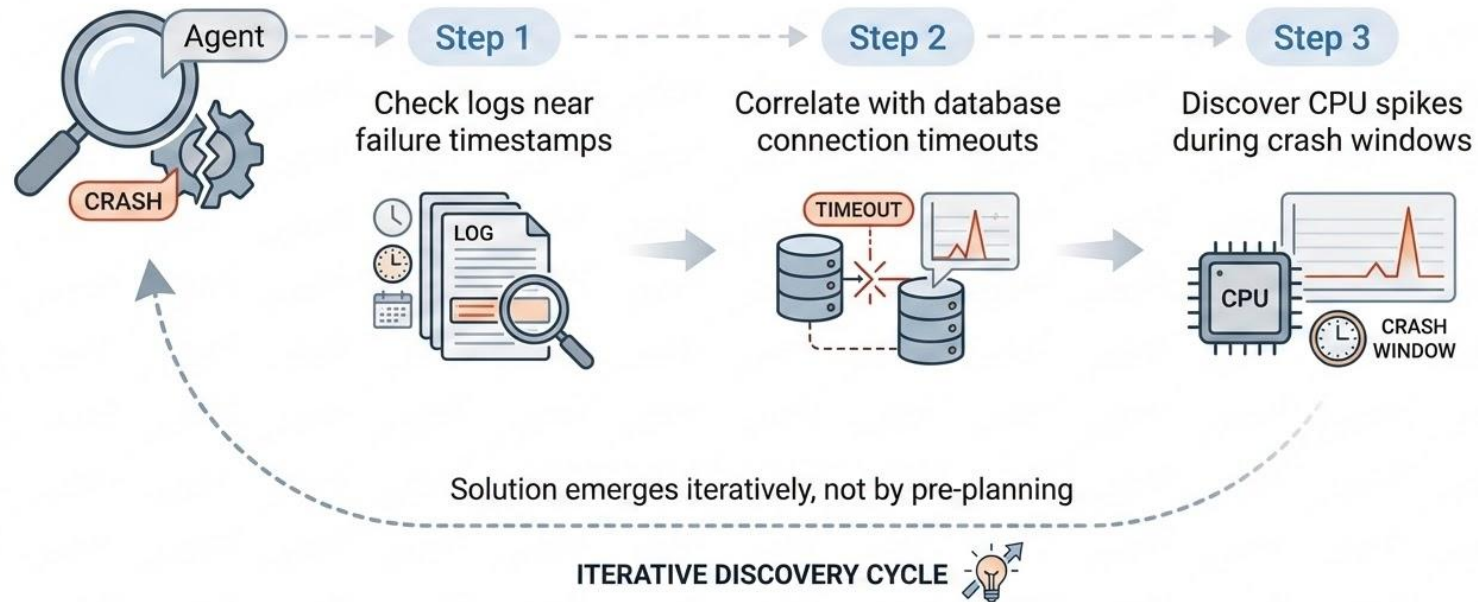
- **Agentic AI is an early and rapidly evolving knowledge domain**
 - Additional resource including [YT videos](#), [references](#), and [study plan](#)
 - Please do the quizzes following the links in the chapter texts (not the study plan)
 - Recorded videos will be sent out asap
 - Continuous Learning credits are still in the work
 - Cohort 2 and the ability to roll back and roll up
- 
- A decorative graphic in the bottom right corner consisting of several overlapping blue rectangular shapes of varying shades, creating a sense of depth and movement.

ReAct Pattern



- Reasoning traces create transparent audit trails
- Grounding through observation prevents hallucination
- Dynamic tool selection adapts to evolving context

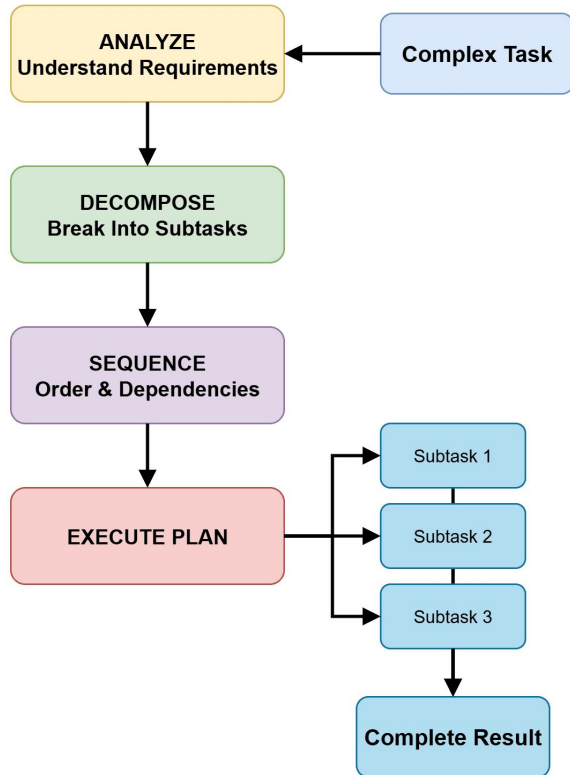
ReAct in Action -- A Debugging Example



ReAct's Hard Truths

- Performance depends on exemplar-query similarity
 - More reasoning steps do not guarantee better results
 - Infinite loops when agents cannot recognize completion
 - Hallucinated reasoning sounds plausible but is wrong
 - Context exhaustion from accumulated history
- 

Plan-and-Execute -- Strategy Before Action

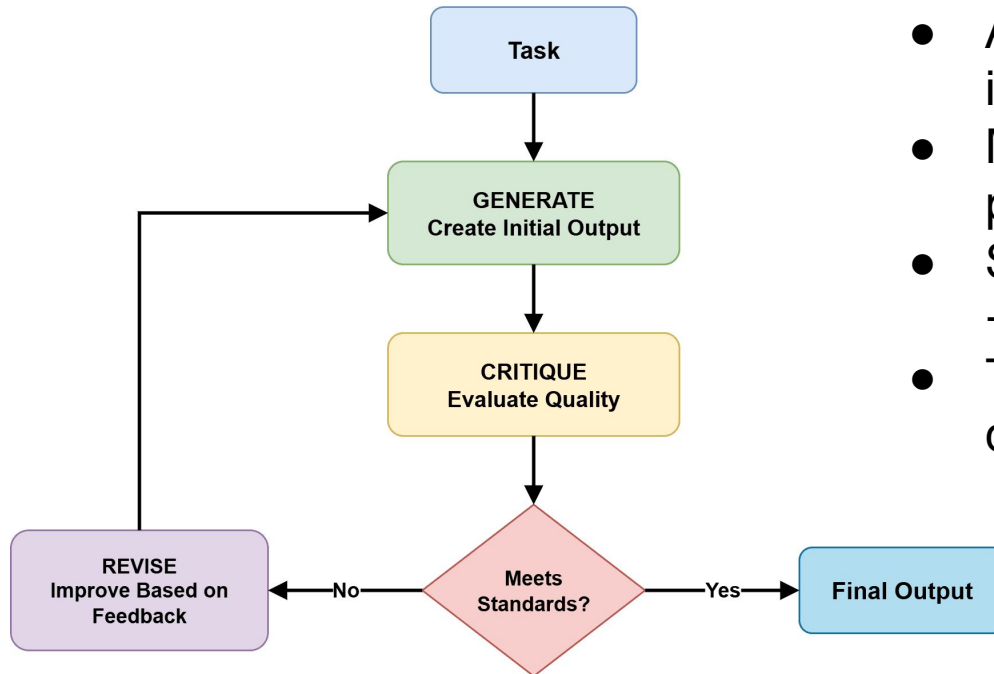


- Separates planning from execution
- Planner decomposes goal into ordered step list
- Executor runs steps using lighter-weight models
- Cost savings of 60-70% vs. ReAct on structured tasks
- Dynamic replanning handles execution failures

Plan-and-Execute Pitfalls

- Plan rigidity fails when assumptions break mid-execution
 - Planning overhead wastes resources on simple tasks
 - Sequential execution misses parallelism opportunities
 - Plan-execution mismatch: planner specifies unavailable tools
 - Cascading failures in multi-agent coordination
- 

Reflection -- The Self-Critique Loop

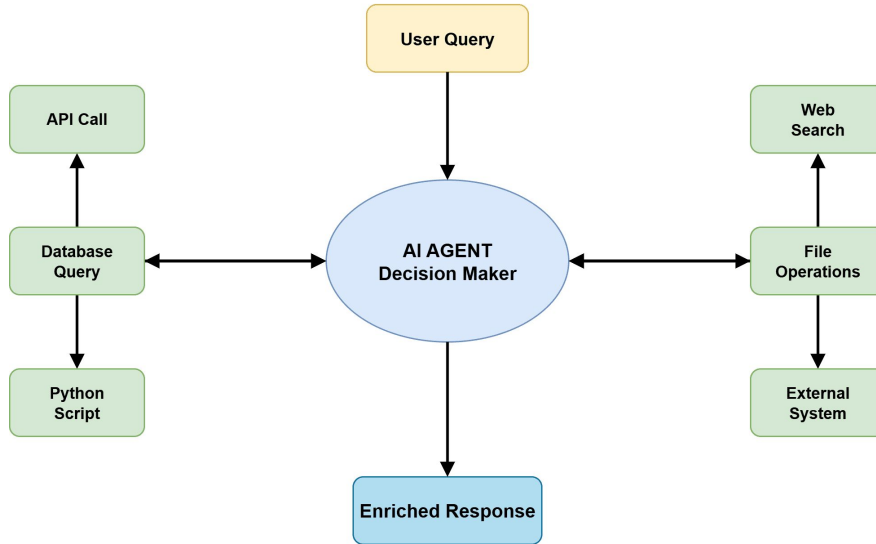


- Generate-Reflect-Refine cycle improves output iteratively
- Agent critiques its own work and identifies weaknesses
- Mimics human draft-review-revise process
- Single-agent or dual-agent (producer + critic) variants
- Triggers: scheduled, post-action, or quality-threshold

Reflection's Diminishing Returns

- Quality plateaus after 2-3 iterations for most tasks
 - Over-correction can replace correct answers with wrong ones
 - Hallucination loops reinforce errors with confident justification
 - Context window fills with critique history, losing original task
 - Self-generated feedback cannot fix knowledge gaps
- 

Tool-Use Architecture



- Extends agents beyond language with function calling
- Tool registry provides metadata, schemas, permissions
- LLM selects tools and constructs structured API calls
- Patterns: REST APIs, webhooks, message queues, sandboxes
- Enables real-time data, deterministic computation, enterprise integration

Tool-Use Traps

- Tool overload: 5-8 focused tools beat 30+ broad ones
 - Tool overuse rate can exceed 50% of all invocations
 - Argument errors: wrong types, missing params, hallucinated names
 - Vague tool descriptions cause misuse and confusion
 - Security risk: prompt injection, privilege escalation
- 

Quiz Review

A developer is building a customer support routing system that categorizes incoming tickets into one of 15 predefined categories based on explicit rules (keywords, customer tier, product type) and then assigns them to the appropriate team queue. Should they use the ReAct pattern for this task?

A:

- No, ReAct is inappropriate here. This is a deterministic, structured workflow with known steps and predefined categories.

Note:

- Pattern selection should match task characteristics.
- 

Quiz Review

Your Plan-and-Execute agent for e-commerce order processing generates excessively detailed plans with 45 steps for tasks that should take 8-10 steps. Analysis shows many steps are independent (inventory checks, payment validation, shipping calculation don't depend on each other). How should you optimize this workflow?

A:

- Constrain planner prompt to generate 8-12 high-level steps, add examples of appropriate granularity, penalize redundant decomposition. Implement DAG-based parallel execution, use async/await for concurrent operations, maintain dependencies only where necessary.

Note:

- Watch out for compound problems requiring integrated solution
- 
- A decorative graphic in the bottom right corner consisting of several overlapping blue rectangular shapes of varying shades, creating a sense of depth and movement.

Quiz Review

A data engineering team needs an agent for ETL pipeline construction. The task has two phases: (1) initial data profiling and schema detection (follows predictable steps: sample data, infer types, detect nulls, identify keys), followed by (2) data quality issue investigation (unpredictable problems like encoding errors, referential integrity violations, outliers). Should they use ReAct or Plan-and- Execute?

A:

- Use Plan-and-Execute for Phase 1, then switch to ReAct for Phase 2 as Phase 2's unknown problems need ReAct's flexibility.

Note:

- Select the right patterns for sub-tasks
- 
- A decorative graphic in the bottom right corner consisting of several overlapping blue rectangular shapes of varying shades, creating a sense of depth and movement.

Think about a complex workflow
which pattern will you choose? Hybrid patterns?

Choosing the Right Pattern

01.

ReAct

unpredictable paths, adaptive investigation needed

02.

Plan-and-Execute

structured workflows, stable environments

03.

Reflection

quality-critical output, code, creative writing

04.

Tool use

external data, computation, enterprise integration

05.

Hybrid

A mix of the above

*Note

Start simple -- add complexity only when evidence demands it

Three Collaboration Paradigms

1 Collaborative

- aligned goals
- peer-to-peer teamwork

2 Competitive

- conflicting interests
- game-theoretic reasoning

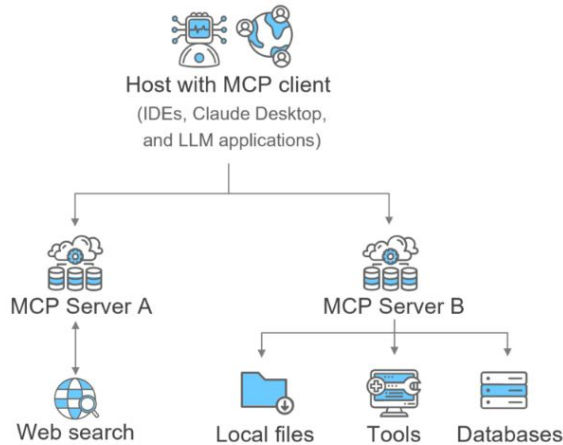
3 Swarm

- simple local rules
- emergent behavior

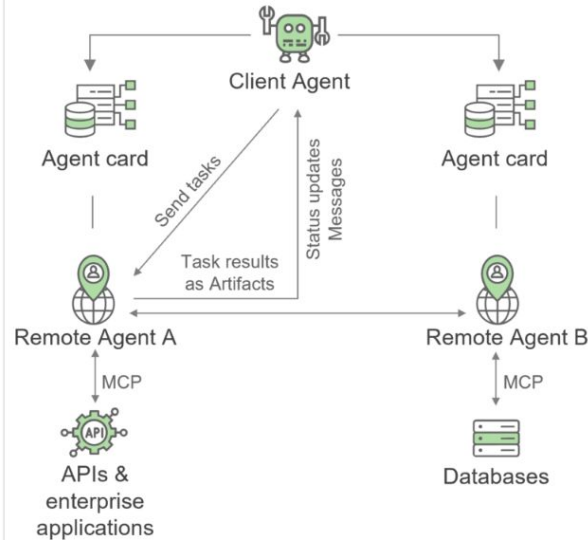
Real systems often blend all three paradigms
Choose paradigm based on agent relationships

Communication Protocols: ACP, A2A, and MCP

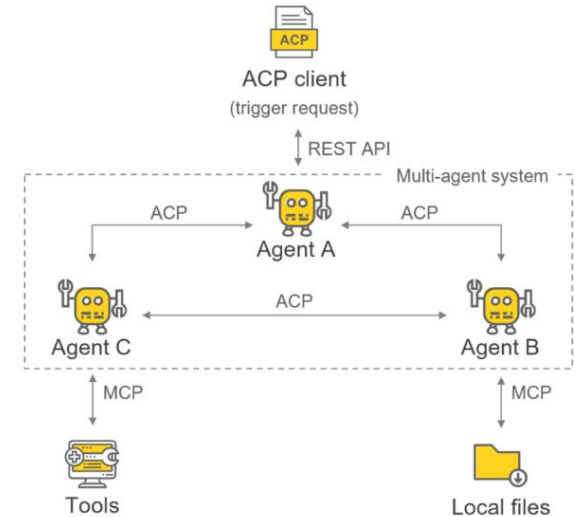
MCP (Model Context Protocol)



A2A (Agent-to-Agent Protocol)



ACP (Agent Communication Protocol)

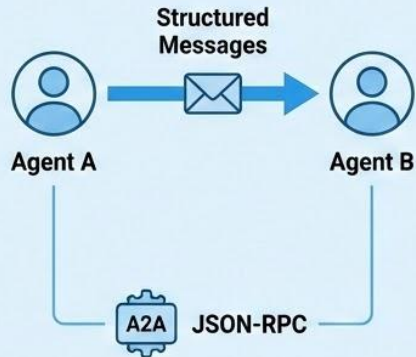


Join AI Tooling Meetup for hands on experience!

Communication Pillars

MESSAGE PASSING

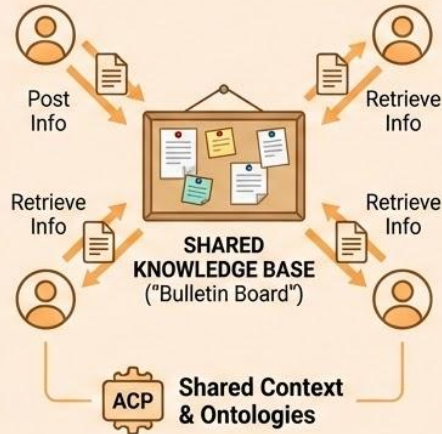
(Direct Point-to-Point)



✓ Agents know endpoints.

BLACKBOARD SYSTEMS

(Shared Knowledge Base)



✓ Asynchronous, no direct awareness.

EVENT-DRIVEN ARCHITECTURES

(Streams of Events/Signals)



✓ Protocols as event streams.

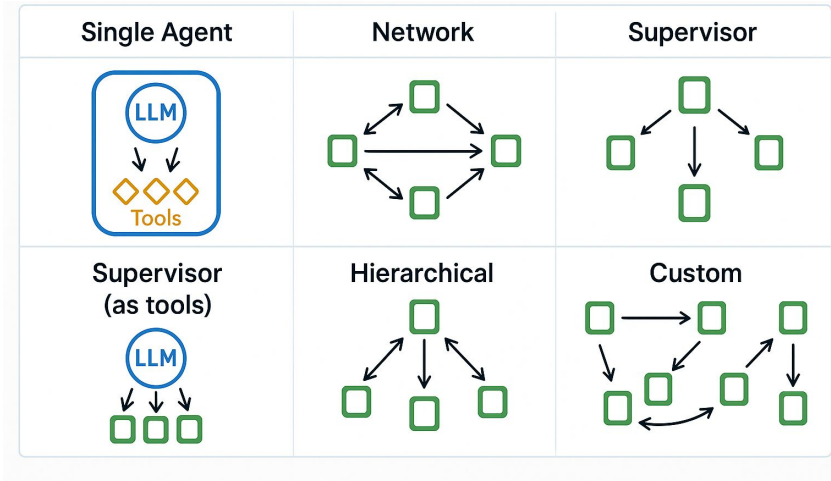
Event-Driven and API-Based Communication

- Publish-subscribe decouples agents via asynchronous events
- Delivery guarantees: at-most-once, at-least-once, exactly-once
- REST offers human-readable, stateless integration
- gRPC provides binary efficiency and bidirectional streaming
- Dead letter queues (DLQ) prevent poison messages from blocking flow

Collaboration Mechanisms Beyond Messaging

- Auction-based allocation: agents bid by workload and confidence
 - Consensus protocols enable decisions without central control
 - Voting mechanisms resolve multi-agent disagreements
 - Shared context requires conflict resolution and versioning
 - Consensus trades optimality for robust decentralization
- 

Orchestration Patterns



- Centralized: single manager, predictable but bottleneck-prone
- Decentralized: peer-to-peer, resilient but hard to debug
- Hierarchical: tiered supervisors balance control and scale
- Federated: autonomous domains coordinate via contracts

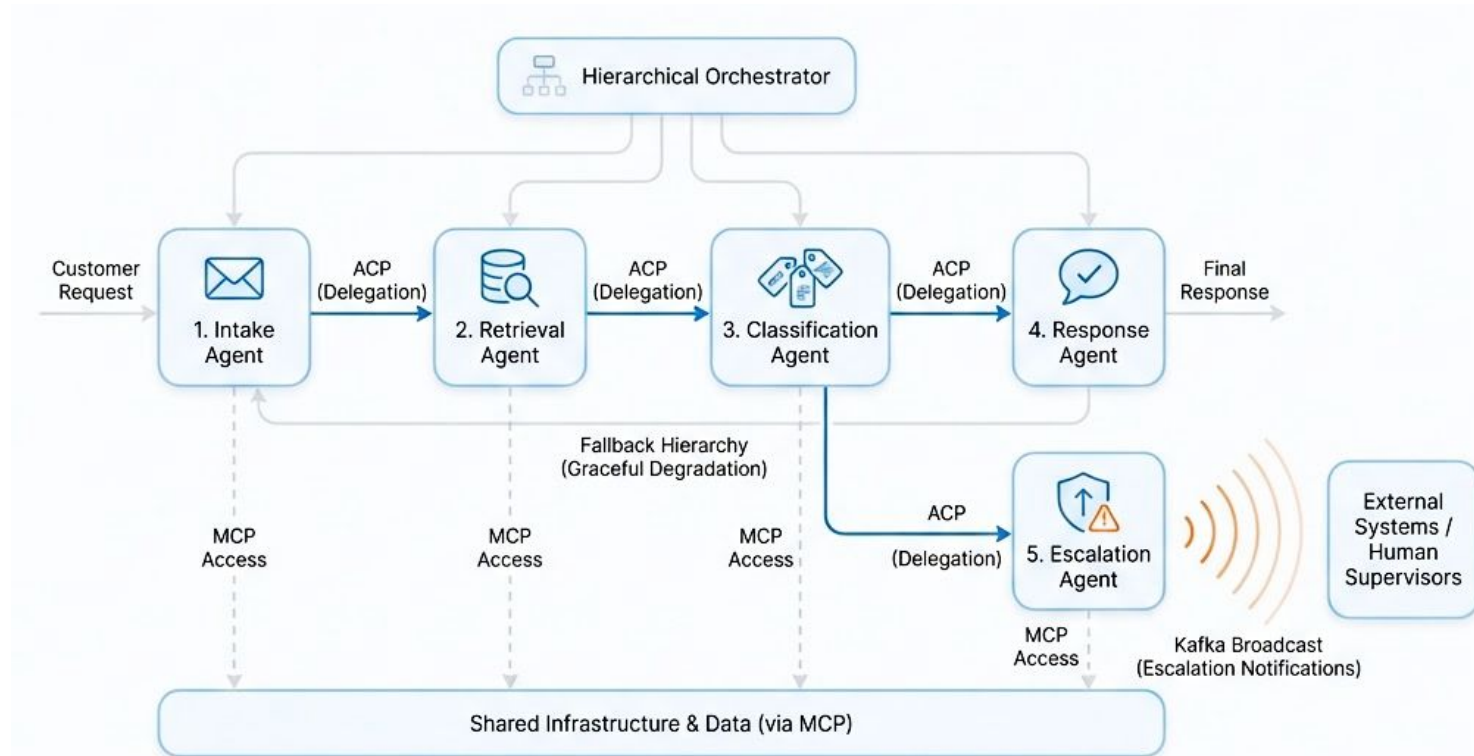
Common Failure Modes in Multi-Agent Systems

- Communication ambiguity
 - Redundant or circular work
 - Task boundary confusion creates security vulnerabilities
 - Format incompatibilities cascade as cryptic runtime errors
 - Root cause: insufficient explicit coordination specification
- 
- A decorative graphic in the bottom right corner consisting of several overlapping, semi-transparent blue rectangles and triangles, creating a modern, abstract look.

Anti-Patterns to Avoid

- Do not use multi-agent when single-agent suffices
 - Avoid "agent monoliths" that deploy and fail together
 - Match sync vs async to your actual dependency graph
 - Always implement circuit breakers and failure isolation
 - Demand comprehensive observability from day one
- 
- A decorative graphic in the bottom right corner consisting of several overlapping, semi-transparent blue rectangles and trapezoids, creating a modern, abstract look.

Worked Example: Customer Support System



Challenges and Successes?

Critical Recommendation

- Find a complex process/workflows of your interest (either work or personal)
- Identify main steps and sub-steps
- How AI Agents can work towards certain steps?
 - Which reasoning pattern(s)?
 - Which orchestration pattern(s)?
 - What are potential issues?
- How humans can drive the process to better outcomes?

Next Chapters

- **Chapter 1.4: Memory & Perception**

This chapter distinguishes between short-term working memory (context window) and long-term systems (semantic, episodic, procedural), introduces perception pipeline stages, and addresses critical integration challenges including the vector store misconception, temporal synchronization, context degradation, and the "lost in the middle" effect.

- **Chapter 1.5A: Stateful Orchestration - Foundations**

This chapter introduces the Stateful Agent Orchestration Model organizing State Storage, Logic Tree Evaluation, and Execution Engine subsystems, while teaching three architectural principles (separation of state and logic, explicit transitions, and idempotent operations) that enable production-grade reliability.